

2400-DS1AL # Algorithms for Data Science

[Kokpit](#) / [Moje kursy](#) / [2400-DS1AL#2025L#277553](#) / [Nowa sekcja](#) / [Lecture notes](#)

Lecture notes

So far in this course we have designed algorithms for specific problems: sorting, graph search, dynamic programming. In this lecture, we ask a different question: how do we *learn* an algorithm from data? The answer turns out to hinge on a single idea — minimizing a function by following its gradient — which we study in detail here.

Machine Learning as Optimization

In supervised machine learning, we are given a set of training examples

$$(x_1, y_1), \dots, (x_n, y_n)$$

where each

$$x_i$$

is an input (e.g., an image) and

$$y_i$$

is the correct output (e.g., a label). We choose a model family — a parameterized function

$$f_{\theta}(x)$$

— and ask: which choice of parameters

$$\theta$$

makes the model's predictions as accurate as possible?

This question is an optimization problem. We define a **loss function**

$$L(\theta)$$

that measures how bad the predictions are (for instance, the squared error between the predicted and true output). Training the model means finding the

$$\theta$$

that minimizes

$$L(\theta)$$

.

In principle, we could search for the best

$$\theta$$

by trying many values. But the parameter space is huge and continuous — modern neural networks can have millions or billions of parameters — so brute force is completely out of the question. We need an iterative optimization algorithm.

Gradient Descent

The key tool is the *gradient* of the loss function. For a function of one variable, the derivative tells us the local slope. For a function of many variables, the gradient

$$\nabla L(\theta)$$

collects all the partial derivatives and points in the direction of *steepest increase* of

$$L$$

. Since we want to *decrease* the loss, we take a small step in the *opposite* direction:

$$-\nabla L(\theta)$$

.

Definition: Gradient Descent Update Rule

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

where

$$\theta_t$$

are the current parameters,

$$\eta > 0$$

is the **learning rate** (step size), and

$$\nabla L(\theta_t)$$

is the gradient of the loss at the current point.

We repeat this update many times, each time moving the parameters slightly downhill.

Worked example.

Suppose

$$L(w) = (w - 3)^2$$

, so

$$\nabla L(w) = 2(w - 3)$$

. Starting at

$$w_0 = 0$$

with learning rate

$$\eta = 0.5$$

:

- Step 1:

$$\nabla L(0) = -6$$

, so

$$w_1 = 0 - 0.5 \cdot (-6) = 3$$

.

- Step 2:

$$\nabla L(3) = 0$$

, so

$$w_2 = 3$$

.

The minimum

$$w^* = 3$$

is reached in a single step. This is a special property of quadratic losses; in general, convergence requires many iterations.

The learning rate is one of the most important choices in practice. If

$$\eta$$

is too small, training progresses very slowly. If

$$\eta$$

is too large, the update overshoots the minimum and the loss may oscillate or diverge.

Convex and Non-Convex Loss Landscapes

For simple models like linear regression, the loss function is *convex*: its graph has no local minima other than the global minimum. Gradient descent on a convex function always converges to the global minimum, regardless of where we start.

Neural networks, however, give rise to *non-convex* loss landscapes with many local minima and saddle points. Gradient descent may converge to a local minimum rather than the global one, and the outcome can depend heavily on the initialization. In practice, this turns out to be less of a problem than it sounds: local minima found by gradient descent for large neural networks tend to perform well empirically, even if they are not globally optimal.

From Linear Models to Neural Networks

A single neuron computes

$$z = w^\top x + b$$

(a weighted sum of its inputs plus a bias), then applies a nonlinear *activation function*

$$\sigma$$

:

$$a = \sigma(z)$$

. Popular choices include the sigmoid

$$\sigma(z) = 1/(1 + e^{-z})$$

and ReLU

$$\sigma(z) = \max(0, z)$$

. Without nonlinear activations, a multi-layer network would collapse to a single linear function and could not learn complex patterns.

A neural network stacks many such neurons into layers. The loss depends on the parameters through many layers of intermediate computations, which creates a challenge: how do we efficiently compute

$$\nabla L$$

with respect to every parameter?

Backpropagation

The answer is **backpropagation**, which is simply the chain rule of calculus applied systematically from the output back through every layer. If

$$L$$

depends on

$$z_2$$

, which depends on

$$z_1$$

, which depends on

$$\theta$$

, then

$$\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial z_1} \cdot \frac{\partial z_1}{\partial \theta}$$

Each factor is a local derivative computed from values already stored during the forward pass. Training a neural network therefore consists of two passes:

- **Forward pass:** compute the prediction

$$\hat{y}$$

and the loss

$$L$$

, storing all intermediate values.

- **Backward pass:** propagate gradient information from the loss back through each layer, multiplying local derivatives as we go.

Without backpropagation, we would need to recompute the gradient from scratch for each parameter separately – an impossibly expensive operation for large networks.

Worked example.

Consider a minimal network with one hidden unit: input

$$x$$

, hidden activation

$$h = \sigma(w_1 x)$$

, prediction

$$\hat{y} = w_2 h$$

, and squared loss

$$L = \frac{1}{2}(\hat{y} - y)^2$$

. With

$$x = 1$$

,

$$y = 0$$

,

$$w_1 = 1$$

,

$$w_2 = 2$$

,

$$\eta = 0.5$$

:

Forward pass:

$$z_1 = 1, \quad h = \sigma(1) \approx 0.73, \quad \hat{y} = 1.46, \quad L \approx 1.07$$

Output layer gradient:

$$\frac{\partial L}{\partial w_2} = (\hat{y} - y) \cdot h = 1.46 \cdot 0.73 \approx 1.07$$

Hidden layer gradient (three-link chain):

$$\frac{\partial L}{\partial w_1} = (\hat{y} - y) \cdot w_2 \cdot \sigma'(z_1) \cdot x = 1.46 \cdot 2 \cdot 0.20 \cdot 1 \approx 0.58$$

Weight update:

$$w'_2 = 2 - 0.5 \cdot 1.07 = 1.47, \quad w'_1 = 1 - 0.5 \cdot 0.58 = 0.71$$

Both weights moved in the direction that reduces

$$L$$

. The key efficiency gain is that the upstream error signal

$$\delta = \partial L / \partial \hat{y}$$

is computed once and reused for every earlier layer.

A complete training iteration consists of: forward pass → loss → backward pass → update. This cycle is repeated many times.

Stochastic and Mini-Batch Training

When the dataset is large, computing the exact gradient over all

$$n$$

examples at every step — **batch gradient descent** — becomes expensive. Two practical alternatives are:

- **Stochastic gradient descent (SGD):** use a single randomly chosen example per update. Cheap, but the gradient estimate is noisy.

- **Mini-batch gradient descent:** use a small batch of examples (e.g., 32 or 256) per update. This balances gradient quality, hardware efficiency, and memory usage, and is the standard approach in practice.

The noise introduced by mini-batches is not purely harmful: it acts as a form of regularization and can help escape poor local minima.

Practical Training Issues

Vanishing and exploding gradients. As the backward pass multiplies many local derivatives together, the gradient can shrink toward zero (*vanishing*) or grow without bound (*exploding*) as it propagates through many layers. Vanishing gradients cause early layers to learn very slowly; exploding gradients cause numerical instability. Standard remedies include careful weight initialization, gradient clipping, and architectural choices such as residual connections.

Overfitting. A model can minimize the training loss perfectly while performing poorly on new data. Typical tools to combat this include weight decay (L2 regularization), dropout, early stopping, and data augmentation.

Stopping criteria. Training is usually stopped after a fixed number of passes through the data (*epochs*), or when performance on a held-out validation set stops improving. Watching only the training loss is not sufficient: the model may overfit while the training loss continues to fall.

Applications

Gradient-based optimization underlies virtually all modern machine learning:

- **Image classification:** convolutional or transformer networks with millions of parameters are trained by repeated mini-batch updates.
- **Recommendation systems:** embeddings and scoring functions that minimize ranking or prediction loss.
- **Natural language models:** large-scale optimization over enormous datasets to predict the next token or answer questions.
- **Tabular prediction:** churn, fraud, credit risk, demand forecasting — even when tree-based models are strong baselines, gradient-based neural models appear in representation learning and multi-modal pipelines.

Summary

The mental map for this lecture is:

model $\longrightarrow$ loss $\longrightarrow$ gradient $\longrightarrow$ update $\longrightarrow$ repeat

Gradient descent is the standard template for training any differentiable model. Backpropagation makes it computationally feasible for deep networks by applying the chain rule in a single backward pass. Mini-batch SGD is the practical workhorse. And as with every algorithm in this course, making it work well requires both mathematical understanding and careful engineering.

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 23:07

[◀ Lecture 15](#)

Przejdź do...

[Lab 1 ▶](#)

✉ Skontaktuj się z pomocą techniczną

Jesteś zalogowany(a) jako Ondřej Marvan (Wyloguj)

2400-DS1AL#2025L#277553